



Guide pour la comparaison des algorithmes

Projet RO



Qu'est-ce que l' Expérimentation des algorithmes?

- **Expérimentation** : c'est la mise en œuvre et l'analyse d'algorithmes pour explorer leur comportement dans des conditions variées.
- **Expérimentation** permet d'évaluer objectivement les performances des algorithmes sur des critères clés tels que le temps d'exécution, la consommation mémoire et la qualité des solutions.



Quesque le Benchmarking?

- **Benchmarking** désigne un processus structuré qui compare plusieurs méthodes en utilisant des références (benchmarks) standardisées. Il est axé sur l'évaluation comparative, souvent avec des métriques prédéfinies pour mesurer les performances.
- Choisir l'algorithme le plus adapté aux spécificités du problème et aux contraintes pratiques, garantissant ainsi une solution efficace et optimale.

Les étapes de l'experimentation

1. Préparer le benchmark de données
2. Générer le code des algorithmes
3. Exécuter les algorithmes
4. Visualiser et Analyser les résultats
5. Interpréter les résultats

Les étapes de l'experimentation

- 1. Préparer le benchmark de données**
2. Générer le code des algorithmes
3. Exécuter les algorithmes
4. Visualiser et Analyser les résultats
5. Interpréter les résultats

Étape 1 : Préparer le benchmark de données

1.Choix des données :

1. Générer automatiquement des instances du problème avec des tailles et paramètres variés.
2. Télécharger des benchmarks standard disponibles en ligne (exemple : TSPLIB pour les problèmes de voyageur de commerce).
3. Utiliser des bibliothèques spécialisées qui contiennent des jeux de données adaptés.

2.Validation des données :

- Vérifier que les données couvrent différents scénarios : petites tailles ($n=10, 25, 50, \dots$), moyennes ($n=100, 150, 200, \dots$), grandes ($n=10^5, 10^7, 10^{10}, \dots$).
- Uniformiser le format des données pour les algorithmes.

Étape 1 : Préparer le benchmark de données

You

a function to generate dataset of the problem VRP with small, medium and big size

BLACKBOXAI

The Vehicle Routing Problem (VRP) involves finding the optimal routes for a fleet of vehicles to deliver goods to a set of customers. To generate datasets for VRP of varying sizes (small, medium, and large), we can create a function that generates random coordinates for customers and a depot, along with the demand for each customer.

Here's a Python function that generates such datasets:

```
python Verify Open In Editor ▶ 📄  
  
1 import random  
2 import pandas as pd  
3  
4 def generate_vrp_dataset(size='small', num_vehicles=3, max_demand=10,  
5     """  
6     Generate a VRP dataset with random customer coordinates and demand
```

Exemple de Prompt IA pour générer des données :

Génère un ensemble de données pour un problème de sac à dos avec 50 objets, chacun ayant un poids entre 1 et 20 et une valeur entre 10 et 100. Le poids maximal du sac est de 100.

Crée une fonction Python `generate\_knapsack\_instance(n\_items, max\_weight)` qui génère une liste de `n\_items` objets, chacun avec un poids et une valeur aléatoires. Le poids total autorisé du sac est `max\_weight`.

Pour chaque prompt :

- modifier les paramètres pour tester différentes tailles (n = 10, 100, 1000...).
- sauvegarder les données dans un fichier .csv ou .json.
- Ajouter une visualisation simple avec matplotlib pour valider la structure des données.

Les étapes de l'experimentation

1. Préparer le benchmark de données
- 2. Générer le code des algorithmes**
3. Exécuter les algorithmes
4. Visualiser et Analyser les résultats
5. Interpréter les résultats

Étape 2 : Générer le code des algorithmes

1. Outils à utiliser :

1. Utiliser des outils de génération de code intelligents (Claude, Blackbox, ChatGPT, Gemini, Copilot, etc.) pour produire les implémentations des algorithmes.
2. Fournir une description claire de chaque algorithme à l'outil, par exemple :
 1. **Algorithmes exacts** : Branch and Bound, Dynamic Programming,....
 2. **Heuristiques** : Greedy Algorithm, Tabu Search,....
 3. **Méthodes métaheuristiques** : Genetic Algorithm, Particle Swarm Optimization,....

```
{  
  "algorithm": "Greedy Knapsack",  
  "execution_time_sec": 0.0021,  
  "memory_usage_kb": 512.3,  
  "solution_value": 187,  
  "solution_valid": True  
}
```

2. Personnalisation du code :

1. Adapter chaque algorithme aux formats des données.
2. Ajouter des métriques de suivi pour le temps d'exécution, l'utilisation mémoire, la qualité de la solution...

Étape 2 : Générer le code des algorithmes

Prompt IA pour générer un algorithme exact :

Implémente l'algorithme Branch and Bound pour résoudre le problème du voyageur de commerce. Utilise une matrice de distances comme entrée.

Prompt IA pour une heuristique :

Implémente une heuristique gloutonne pour le problème du sac à dos. Trie les objets par rapport valeur/poids.

Prompt IA pour ajouter des métriques d'évaluation

Ajouter des métriques directement dans le code :
`time, memory_profiler, solution_quality.`

Structure générale du prompt

Implémente l'algorithme [nom] en Python pour résoudre le problème [type de problème]. Le code doit :

- Accepter des données en entrée sous forme de [format attendu].
- Mesurer le temps d'exécution avec ``time``.
- Mesurer la mémoire utilisée avec ``tracemalloc`` ou ``memory_profiler``.
- Retourner la solution trouvée et sa qualité (valeur objective).
- Afficher ou enregistrer les métriques dans un fichier ou un tableau.

Les étapes de l'experimentation

1. Préparer le benchmark de données
2. Générer le code des algorithmes
3. **Exécuter les algorithmes**
4. Visualiser et Analyser les résultats
5. Interpréter les résultats

Étape 3 : Exécuter les algorithmes



1. Configurations à considérer :

1. Exécuter les algorithmes sur plusieurs tailles de données pour observer l'évolution de leurs performances.
2. Répéter plusieurs fois pour calculer des statistiques fiables (moyennes, écarts types).

2. Mesures à capturer :

1. Temps d'exécution (en secondes ou millisecondes).
2. Espace mémoire utilisé (en Mo ou Ko).
3. Qualité de la solution obtenue :
 1. Pour les problèmes de minimisation : valeur minimale trouvée.
 2. Pour les problèmes de maximisation : valeur maximale trouvée.

3. Automatisation des tests :

1. Créer un script Python pour automatiser l'exécution de tous les algorithmes sur les données.
2. Afficher les résultats dans la console ou enregistrer dans un fichier.

Étape 3 : Exécuter les algorithmes



Script Python type :

```
for algo in algorithmes:  
    for dataset in jeux_de_donnees:  
        resultats = algo.run(dataset)  
        log(resultats)
```

Prompt IA pour automatiser les tests :

Crée un script Python qui exécute plusieurs algorithmes sur des jeux de données et enregistre le temps d'exécution, la mémoire utilisée et la qualité de la solution dans un fichier CSV.



Les étapes de l'experimentation

1. Préparer le benchmark de données
2. Générer le code des algorithmes
3. Exécuter les algorithmes
4. **Visualiser et Analyser les résultats**
5. Interpréter les résultats

Étape 4 : Analyser les résultats

1.Création des tableaux :

Construire des tableaux comparant les performances des algorithmes pour chaque jeu de données.

2.Visualisation :

Utiliser des bibliothèques comme Matplotlib ou Seaborn pour tracer des graphiques :

- 1.Comparaison des temps d'exécution.
- 2.Consommation mémoire.
- 3.Qualité des solutions.

3.Mesures statistiques :

Calculer la moyenne, le pourcentage d'écart avec l'optimal connu, le taux de réussite (solution valide trouvée vs non trouvée).

Étape 4 : Visualiser et Analyser les résultats

Prompt IA pour visualisation :

Génère un graphique comparatif des temps d'exécution de trois algorithmes sur des instances de tailles différentes. Utilise Matplotlib.



Les étapes de l'experimentation

1. Préparer le benchmark de données
2. Générer le code des algorithmes
3. Exécuter les algorithmes
4. Visualiser et Analyser les résultats
- 5. Interpréter les résultats**

Étape 5 : Interpréter les résultats

1. Analyse comparative :

Identifier l'algorithme le plus performant selon :

1. Les critères d'efficacité (temps d'exécution, espace mémoire).
2. La qualité des solutions.
3. La robustesse sur des jeux de données variés.



2. Synthèse :

1. Justifier le choix de l'algorithme optimal.
2. Identifier les limitations de chaque approche (par exemple, faible performance sur des instances de grande taille).